

Frontend Development Internship

Task 3: NovaCalc — JavaScript DOM Calculator

Project Documentation

Project Name	NovaCalc
Project Type	JavaScript DOM Fundamentals Calculator
Live Website	https://novaacalc.netlify.app/
GitHub Repository	https://github.com/04nandini0/nova-calc

1. Project Overview

NovaCalc is a browser-based scientific calculator application, branded as "NOVA SCIENTIFIC v2.4" with the tagline "Calculate Beyond Numbers — Precision. Power. Simplicity." It provides both standard arithmetic operations and an extended set of scientific functions (trigonometric, logarithmic, power, and factorial operations) within a single, self-contained interface.

The purpose of building NovaCalc was to move beyond a basic four-function calculator and practice implementing a more complete, real-world calculator application using core JavaScript DOM fundamentals — capturing button-driven input, evaluating expressions, updating the display in real time, and managing application state such as calculation history, angle mode, and sound/theme preferences, entirely on the frontend.

As a JavaScript DOM fundamentals project, NovaCalc demonstrates practical application of element selection, event handling, and dynamic content updates: every button press is captured by an event listener, reflected immediately in the display, and — where applicable — used to update the persisted history panel, all without a page reload.

2. Project Objective

- Understand and apply DOM manipulation to dynamically update the calculator display and history panel in response to user actions.
- Handle user interactions through click-based input across a large grid of standard and scientific operation buttons.
- Practice JavaScript logic for expression evaluation, including operator precedence and scientific functions such as trigonometry, logarithms, powers, and factorials.
- Build a fully interactive frontend application with real-time feedback, including error handling for invalid operations (e.g., undefined results).
- Implement supporting features such as calculation history persistence, angle mode switching (RAD/DEG), and user-configurable sound and theme preferences.

3. Technologies Used

NovaCalc is built entirely using core frontend web technologies, with all interactivity handled through vanilla JavaScript.

Technology	Purpose
HTML5	Structure for the calculator display, standard and scientific button grids, header controls, and the history panel.
CSS3	Dark-themed visual styling, button grid layout, colour-coded operator/function buttons, and responsive layout adjustments.
JavaScript (Vanilla)	Core calculator logic — capturing button input, evaluating expressions, updating the display, handling scientific functions, managing calculation history, RAD/DEG mode, sound toggle, and theme switching.

Technology	Purpose
Netlify	Deployment and hosting of the live web application.

4. JavaScript DOM Concepts Implemented

4.1 Selecting DOM Elements

The calculator relies on selecting and referencing key DOM elements — the display/output area, each operation and digit button, the RAD/DEG toggle, and the history panel — in order to read and update their state.

4.2 Event Listeners and Handling Button Clicks

Each calculator button (digits, operators, scientific functions, AC, DEL, and equals) is wired to an event listener that captures the click and triggers the corresponding logic, as seen in the button highlight states visible when a button is pressed (e.g., the DEL button shown highlighted in red, and operator buttons highlighted in purple when active).

4.3 Updating Webpage Content Dynamically

The display updates in real time as the user builds an expression (e.g., showing "21+9" as it is typed) and then updates again to show the evaluated result (e.g., "21+9 =" followed by "30"), all without reloading the page.

4.4 Managing User Input

User input is accumulated as an expression string built from sequential button presses, supporting digits, decimal points, parentheses, and both basic and scientific operators.

4.5 Performing Calculations

On pressing the equals button, the accumulated expression is evaluated and the numeric result is displayed. The application also handles invalid operations gracefully, displaying an "Error" state (observed as "cube(NaN) = Error") when a calculation cannot be resolved.

4.6 Dynamic History Management

Completed calculations are dynamically appended to a History panel, each entry showing the original expression and its result (e.g., "21+9" → "30"). This confirms the application persists calculation history rather than only displaying it as static UI, along with a "Clear all" control to reset the saved history.

5. Features Implemented

- Standard arithmetic operations: addition, subtraction, multiplication, and division.
- Scientific functions: $\sin$, $\cos$, $\tan$, $\sin^{-1}$, $\cos^{-1}$, $\tan^{-1}$, $\log$, $\ln$, $\sqrt{}$ (square root), $|x|$ (absolute value), π , e , x^2 , x^3 , x^y (power), $n!$ (factorial), $1/x$ (reciprocal), and $\bmod$ (modulus).
- Parentheses support (and) for grouped expressions.
- AC (all clear) and DEL (delete last entry) controls.

- Percentage (%) operation.
- RAD / DEG toggle for switching the angle mode used in trigonometric calculations.
- Dynamic display updates showing both the in-progress expression and the final evaluated result.
- Error handling for invalid operations, displaying a clear "Error" state rather than an incorrect or blank result.
- Persistent calculation history panel that logs each expression and its result, with a "Clear all" option.
- A sound toggle control that turns keypress and result sound effects on or off.
- Theme colour selector offering multiple accent colour options (shown as four selectable colour dots — purple, green, orange, and teal).

6. User Interface Description

6.1 Calculator Layout

The NovaCalc interface is presented as a self-contained calculator card centred on a dark, starfield-styled background. The card is headed by the "NOVACALC" label and the current angle mode indicator (RAD), followed by the display area, and then the button grid below it.

6.2 Button Arrangement

Buttons are arranged in two logical groups: an upper 4-row grid of scientific functions (sin, cos, tan, log, ln, inverse trig functions, $\sqrt{}$, $|x|$, π , e, powers, factorial, reciprocal, mod, and parentheses), and a lower grid of standard calculator keys (AC, DEL, %, $\div$, $\times$, $-$, $+$, digits 0–9, decimal point, and a full-width equals button). Operator buttons are visually distinguished with a purple highlight, and the equals button uses a distinct blue-to-purple gradient to stand out as the primary action.

6.3 Display Area

The display area shows the current expression as it is typed, right-aligned, and — once evaluated — shows the original expression in smaller text above the final result shown in a larger font size (e.g., "21+9 =" above "30").

6.4 User Interaction Flow

A user builds an expression by pressing digit and operator buttons, watching the expression appear live in the display, and then presses equals to see the evaluated result. The result and originating expression are automatically added to the History panel, which the user can open via a dedicated history icon in the header and clear at any time.

6.5 Responsive Behavior

At mobile viewport widths, the calculator card scales down to fit the narrower screen while retaining the full button layout, header branding, and controls, as confirmed by the mobile screenshot showing the complete interface — including an active error state — rendered correctly on a narrow screen.

7. Development Process

7.1 Designing Calculator Structure

The project began by planning the calculator's core structure: a display area, a set of scientific function buttons, a set of standard operation buttons, and supporting controls (history, sound, theme, angle mode).

7.2 Creating HTML Layout

HTML was used to lay out the calculator card, the display element, and the full grid of scientific and standard buttons, along with the header controls (theme dots, sound toggle, history toggle).

7.3 Styling the Interface

CSS was used to apply the dark, starfield-themed visual design, style the calculator card and button grid, colour-code operator and function buttons, and ensure the layout adapted responsively to smaller screens.

7.4 Implementing JavaScript Logic

JavaScript was used to implement expression building from button input, evaluate the resulting expression (including scientific functions and RAD/DEG-aware trigonometry), and handle error cases such as invalid or undefined results.

7.5 Connecting DOM Elements

Event listeners were attached to every button to capture user input, update the display element in real time, and push completed calculations into the history panel's DOM structure.

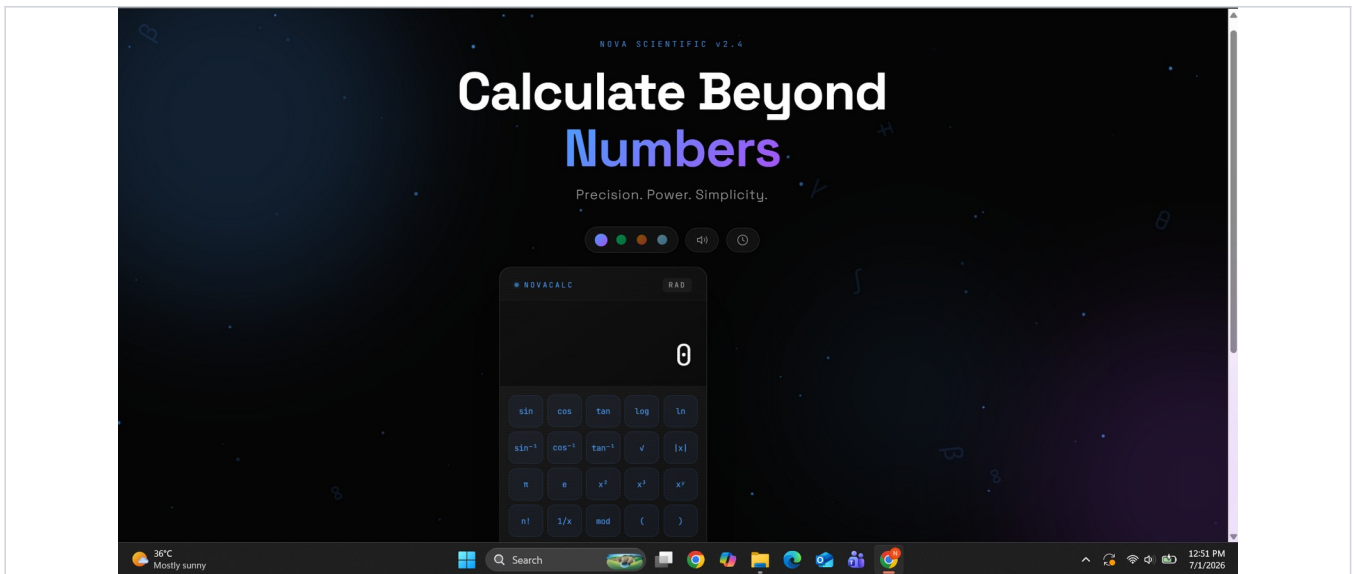
7.6 Testing Functionality

The calculator was manually tested by performing standard arithmetic (e.g., $21 + 9 = 30$) and scientific operations, verifying the history panel correctly logged multiple calculations (including negative and decimal results), confirming the RAD/DEG toggle, sound toggle, and theme selector all responded correctly, and checking that invalid operations produced a clear Error state rather than a silent failure.

8. Screenshots Documentation

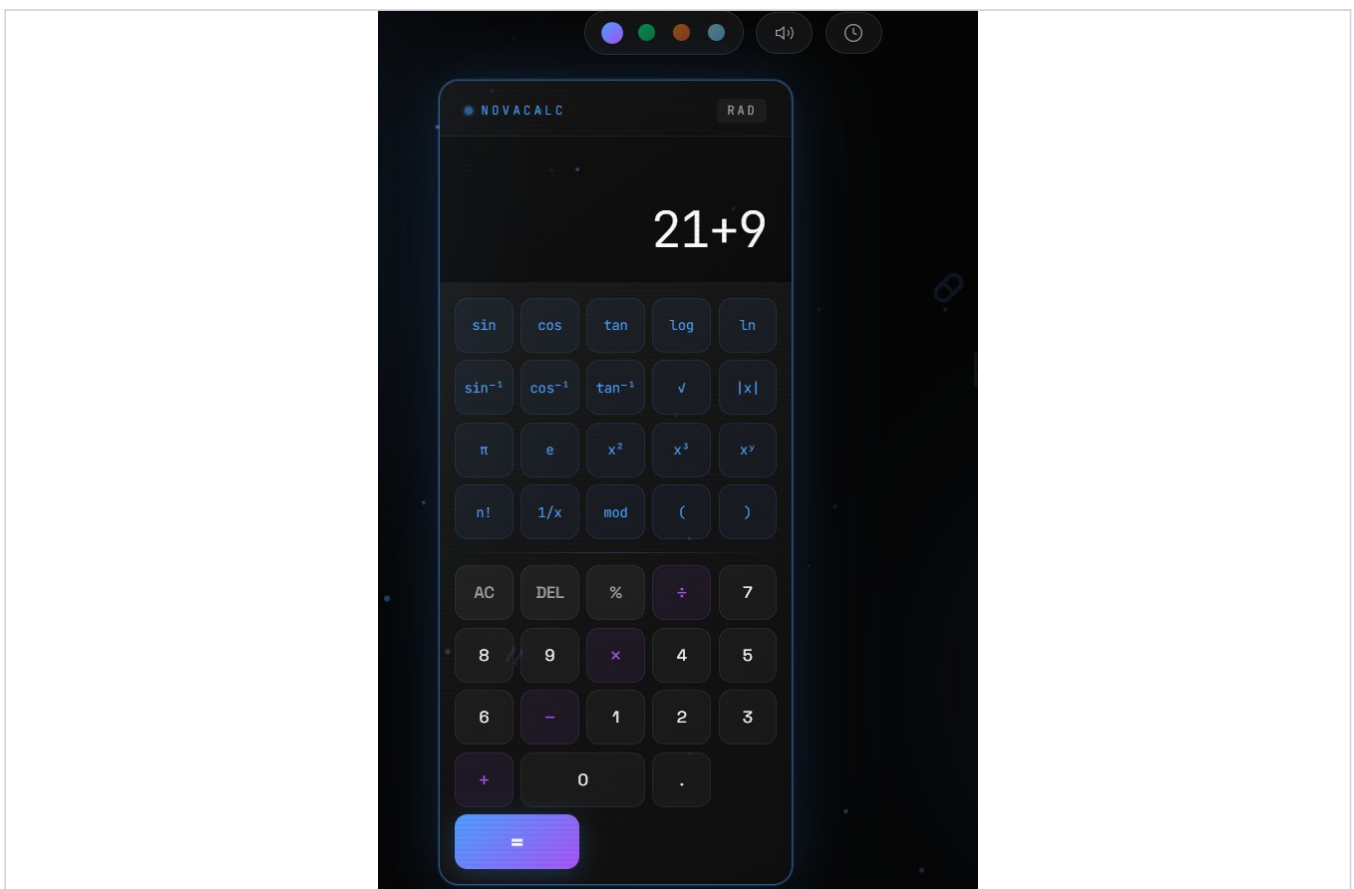
The following screenshots were captured directly from the live, deployed NovaCalc application.

Figure 1: Main Calculator Interface



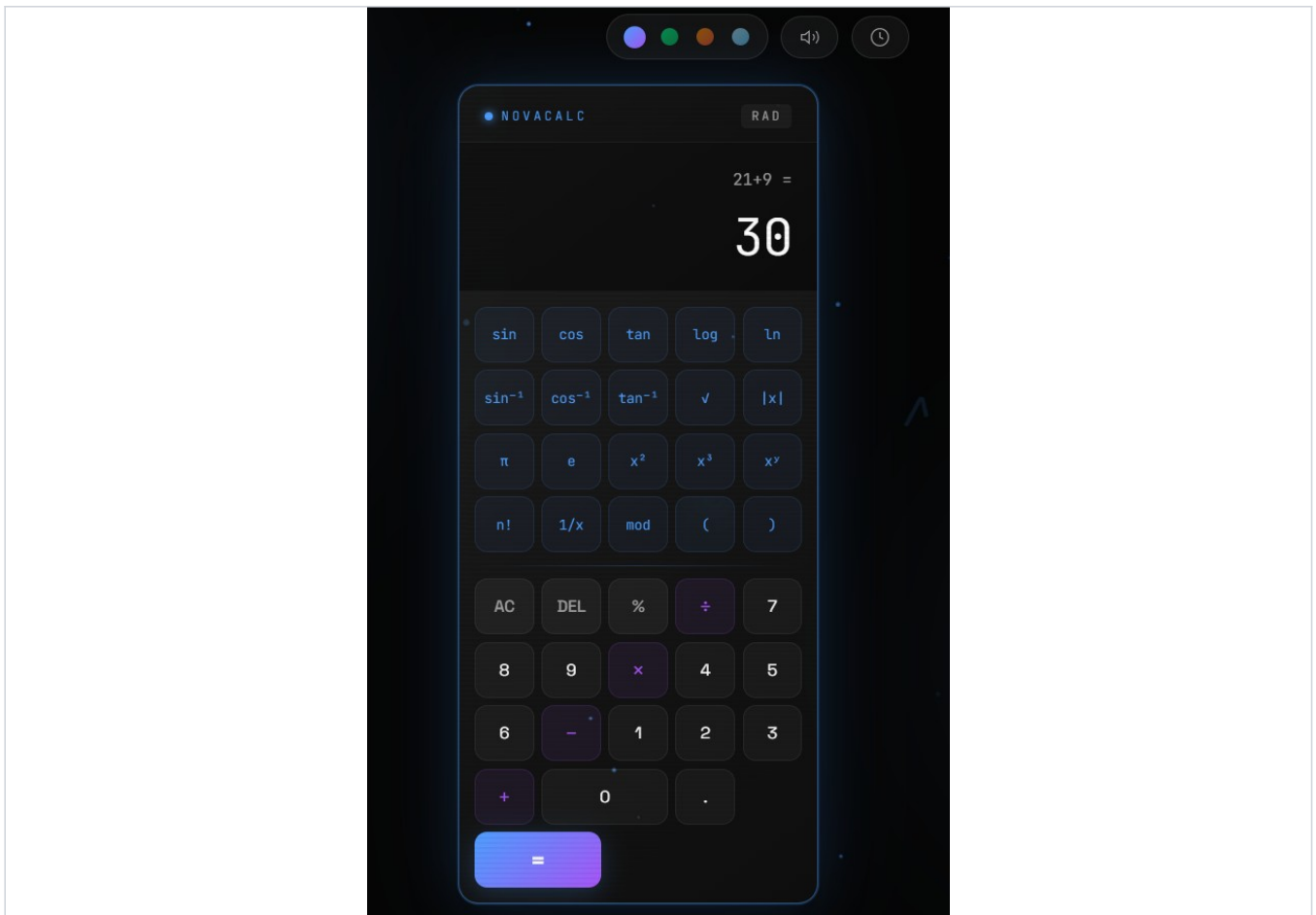
Shows the NovaCalc landing view with the "Calculate Beyond Numbers" branding, theme and control icons, and the calculator card in its default state — display reading 0, RAD mode active, and the scientific function grid visible.

Figure 2: Calculation Being Performed



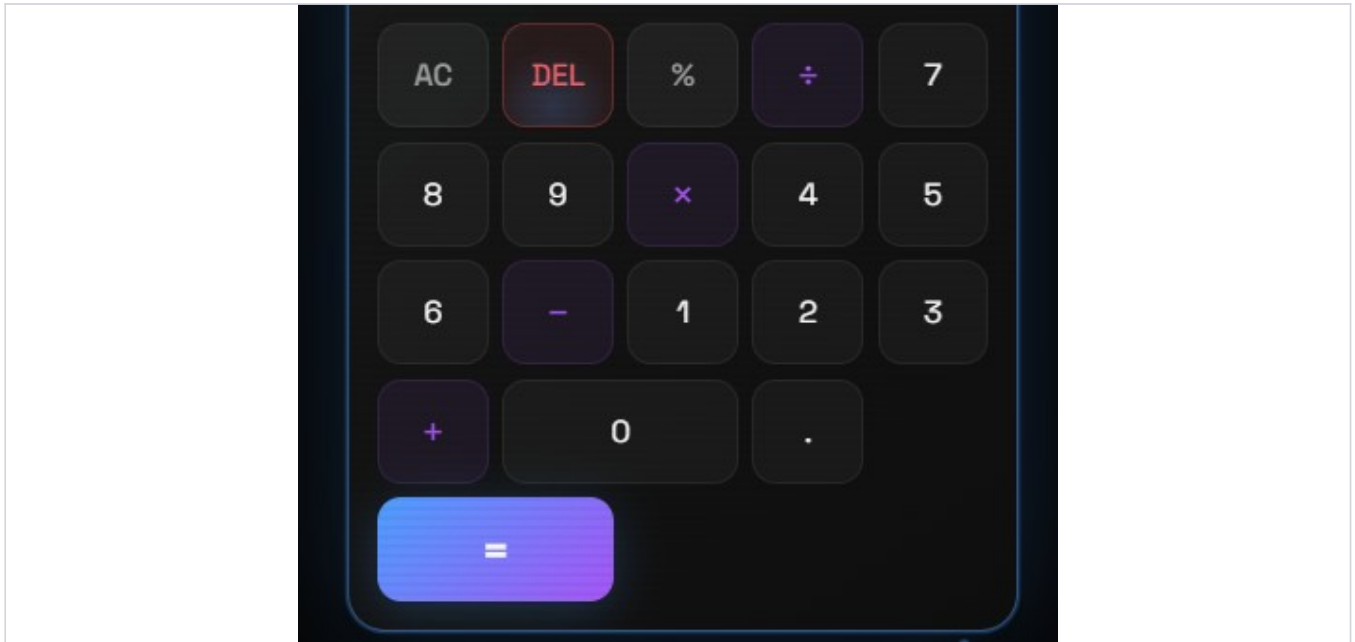
Shows the display updating dynamically as the expression "21+9" is entered, with the active operator button (+) highlighted, demonstrating real-time DOM updates driven by button click events.

Figure 3: Display/Output Section Showing Result



Shows the calculator after pressing equals: the original expression "21+9 =" is displayed above the evaluated result "30", demonstrating the calculation and display-update logic.

Figure 4: Buttons and User Interaction Area



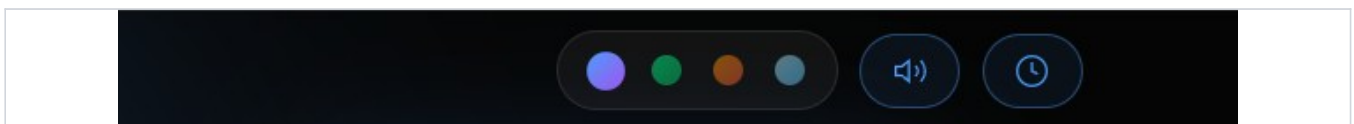
A close-up of the standard operation button grid, showing the AC, DEL, %, operator, digit, and equals buttons, with the DEL button highlighted to indicate an active/hovered interaction state.

Figure 5: Calculation History Feature



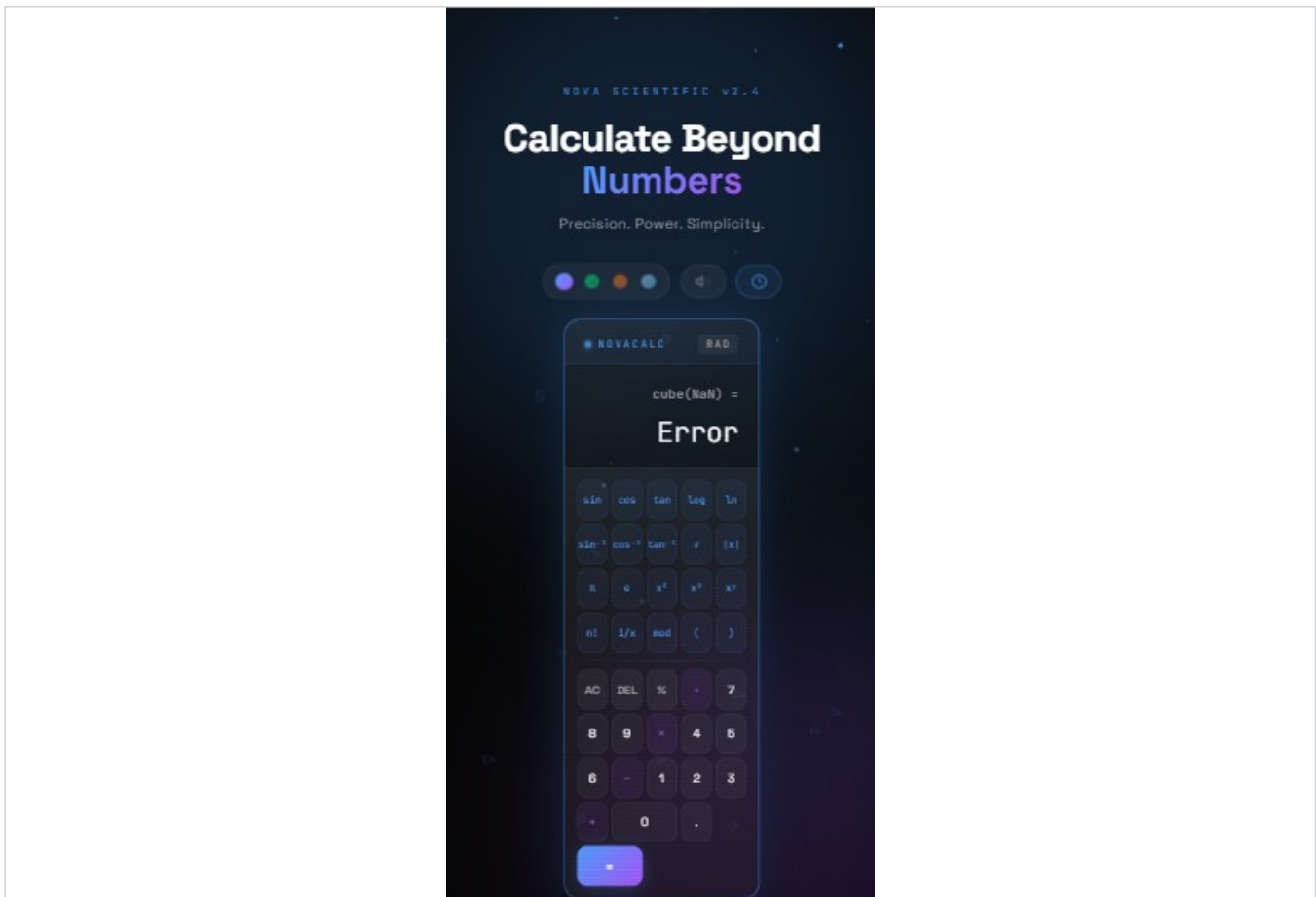
Shows the History panel opened via the header's history icon, listing multiple previously performed calculations (expression and result pairs) along with a "Clear all" control, confirming that calculation history is dynamically stored and displayed.

Figure 6: Header Controls — Theme, Sound & History



A close-up of the header control row: four selectable theme colour dots, a sound toggle that turns keypress and result sound effects on or off, and the history panel toggle.

Figure 7: Mobile Responsive View with Error Handling



Shows the full NovaCalc interface rendered at a mobile viewport width, including the header branding, theme/sound/history controls, and button grid, while the display demonstrates the application's error handling with "cube(NaN) = Error" shown for an invalid calculation.

9. Challenges Faced

- Connecting a large number of individual buttons (standard and scientific) to JavaScript logic while keeping the expression-building and evaluation code organized and maintainable.
- Managing user input accurately as an expression string, including handling operators, decimal points, and parentheses in the correct sequence.
- Implementing scientific calculation operations correctly, including making trigonometric functions respect the active RAD/DEG mode.
- Handling invalid or undefined calculation results (such as operations producing NaN) gracefully, by detecting and displaying a clear Error state instead of a broken or misleading output.
- Updating the UI dynamically in multiple places at once — the main display, the active-button highlight state, and the history panel — while keeping them all in sync with the underlying calculator state.

10. Key Learnings

- Practical, hands-on experience with core JavaScript fundamentals, including variables, functions, and conditional logic for evaluating expressions.

- Strong grounding in DOM manipulation — selecting elements, reading and writing their content, and reflecting application state changes in the UI immediately.
- Event handling experience, wiring up click listeners across a large set of interactive buttons and responding appropriately to each one.
- Frontend problem solving, particularly around structuring calculator logic (expression building, evaluation, and error handling) in a way that stays reliable across many possible input combinations.
- Experience building a genuinely interactive application with persisted state (calculation history) and user preferences (sound, theme, angle mode), going beyond a static UI.

11. Future Improvements

- Add keyboard support so calculations can be entered and evaluated using the physical keyboard, not just on-screen buttons.
- Improve UI animations, such as smoother transitions when switching themes or when a new result is displayed.
- Add advanced mathematical operations, such as additional statistical functions, unit conversions, or memory (M+/M-/MR) support.
- Allow individual history entries to be re-used (tapped to reload into the display) rather than only viewed.
- Add export or copy functionality for calculation history.
- Enhance accessibility with ARIA labelling and full keyboard navigability across all buttons.

12. Project Links

Live Demo	https://novaacalc.netlify.app/
GitHub Repository	https://github.com/04nandini0/nova-calc